

Campus Resource & Event Portal

Submitted in partial fulfillment of the requirements of the degree of

BACHELOR OF COMPUTER ENGINEERING

by

Mahika Desai - 24102033

Aarya Bhoir - 24102047

Neha Bagal - 24102050

Archita Dedhia - 24102073

Guide:

Prof. Selvin Furtado



Department of Computer Engineering

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

2025-26



Parshvanath Charitable Trust's
A. P. SHAH INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi & Govt. of Maharashtra, Affiliated to University of Mumbai)
(Religious Jain Minority)

CERTIFICATE

This is to certify that the Mini Project entitled “**Campus Resource & Event Portal**” is a bonafide work of “**Mahika Desai(24102033), Aarya Bhoir(24102047), Neha Bagal (24102050), Archita Dedhia(24102073),**” submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Engineering**.

Guide
Prof. Prof. Selvin Furtado

Project Coordinator
Prof. Deepali Kayande

Head of Department
Dr. S.H. Malave



Parshvanath Charitable Trust's
A. P. SHAH INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi & Govt. of Maharashtra, Affiliated to University of Mumbai)
(Religious Jain Minority)

Project Report Approval for Mini Project

This project report, entitled “**Campus Resource & Event Portal**” by *Mahika Desai, Aarya Bhoir, Neha Bagal, Archita Dedhia* is approved for the partial fulfilment of the degree of *Bachelor of Engineering in Computer Engineering, 2025-26*

Examiner Name

Signature

1. _____

2. _____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Mahika Desai – 24102033

Aarya Bhoir – 24102047

Neha Bagal – 24102050

Archita Dedhia – 24102073

Date:

Abstract

The increasing number of campus activities in educational institutions has created challenges in managing event information, communication, and resource allocation effectively. Traditional methods, which rely on fragmented platforms such as emails and messaging applications, often lead to poor coordination, missed announcements, and scheduling conflicts. To address these issues, this project aims to design and develop a centralized Campus Event Management Portal that streamlines event creation, registration, and communication between students and organizers.

The system is developed using a modern web development stack, with React for the frontend, supported by backend services for efficient data handling, and a secure database for reliable storage. It includes key features such as event categorization, email notifications, calendar integration, and real-time updates, ensuring a seamless user experience.

The results of implementing this system show significant improvements in event management efficiency, increased user engagement, and a reduction in duplication and miscommunication. Additionally, the portal enhances transparency by providing detailed reports on registrations and participation metrics. In conclusion, the proposed system offers a scalable, user-friendly solution that improves the organization and accessibility of campus events. It also aligns with Sustainable Development Goals by promoting inclusive access to information and efficient institutional management, specifically supporting SDG 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure), and SDG 16 (Peace, Justice, and Strong Institutions).

Keywords: Campus Event Management, Event Portal, Centralized System, Student Engagement, Event Registration, Notification System, Calendar Integration, Web Application, Resource Management, Digital Transformation.

CONTENTS

Sr. No.	Chapter Name	Page No.
1	Introduction	1
2	Existing System Study	3
3	Problem Statement, Objective & Scope	5
4	Proposed System	7
5	Project Plan	12
6	Experimental Setup Technology Stack Used Database Design (ER Diagram, Schema, Normalization) Module Description)	14
7	Implementation Details (Front end & Database Integration and SQL Queries & Implementation)	18
8	Results (Screenshots)	20
9	Conclusion	30
10	References	31

LIST OF FIGURES

Sr. No.	Figure Name	Page No.
1	Gantt Chart	12
2	Architecture Diagram	07
3	Entity Relationship Diagram	08
4	Data Dictionary	11
5	Sample SQL queries	12
6	Database Constraints (if any)	-

Chapter 1

Introduction

In present-day institutes, managing campus activities, events and shared resources requires coordination among students, faculty and administrative staff. Even though digital tools are commonly available, many processes still run through separate channels such as emails, messaging apps, spreadsheets and manual booking registers. Because these tools are not connected, information is not always synchronized and different stakeholders may see different versions of schedules or bookings. This situation often results in overlapping events, repeated follow-ups and missed notifications.

When there is no unified system, day-to-day workflows become harder to manage. Event organizers must repeatedly check room availability, maintain participant lists, inform students of changes and coordinate with multiple departments. Students and faculty, on the other hand, may not have a single, reliable place to check which events are happening and how to register. These gaps show that a structured, campus-wide platform is needed to support communication, scheduling and basic resource allocation.

This project proposes a Campus Resource & Event Management Portal that acts as that single platform. The system is designed to support key tasks such as creating events, assigning resources, publishing event details and handling registrations through one interface. By doing so, it aims to make campus operations more systematic and to reduce dependence on informal communication methods. The portal follows a modular design that separates user interface, backend services and database storage, improving maintainability and performance.

The frontend is developed using React, a popular JavaScript library for building interactive web interfaces. Its component-based structure makes it easier to design reusable elements such as event cards, forms and dashboards, while efficient state management helps in updating information on the screen without full page reloads. Pages are designed to be responsive so that users can access the portal comfortably from different devices.

On the server side, FastAPI is used to implement the application logic and expose RESTful endpoints. These endpoints handle operations like authentication, event CRUD (Create, Read, Update, Delete), registration handling and resource booking. FastAPI's asynchronous nature and built-in request validation make the system responsive and reliable when multiple users interact with it at the same time.

For data storage, the project uses MySQL, a widely adopted relational database management system. The schema is designed with normalized tables for users, events, categories, registrations, resources and bookings. Relationships and constraints are used to maintain data integrity and avoid inconsistent records. Indexes and simple optimizations are applied to keep query performance reasonable as the number of events and users grows.

Security and access control form an important part of the system design. Authentication ensures that users must log in before accessing sensitive features, and authorization rules restrict critical actions (like creating or modifying events) to specific roles such as admin or staff. Input validation and secure handling of API requests help in protecting data from accidental misuse.

Scalability is also considered. Because the system is based on RESTful APIs and a decoupled frontend, it can be extended in the future with features such as mobile applications or integration with institutional portals. Logging of errors and key actions makes it easier to diagnose issues and maintain the application over time.

By combining these elements into a single solution, the portal aims to simplify campus event management and resource usage. It replaces scattered tools with an integrated workflow, making it easier for organizers to plan and for students to participate. In this way, the project demonstrates how modern web technologies can directly support everyday academic activities.

Chapter 2

Existing System Study

1. Manual Notice Board System

- Events and announcements are displayed on physical notice boards
- No real-time updates or digital access
- Students may miss important information

2. Email-Based Communication

- Event details shared through institutional emails
- Messages can be overlooked or ignored
- No centralized tracking of events or participation

3. Messaging Platforms (WhatsApp/Telegram Groups)

- Used for quick communication of events and updates
- Information gets lost in chats due to high message volume
- No structured event management or record keeping

4. Google Forms & Sheets-Based Management

- Used for event registrations and data collection
- Requires manual handling of responses and updates
- Lacks automation and integration with event scheduling

5. Basic College Websites

- Static pages used to display event information
- Limited interactivity and no real-time updates
- No user-specific features like registration or dashboards

Key Limitations of Existing Systems

- Lack of centralization
- Poor communication and coordination
- No role-based access control
- High manual workload
- Increased chances of errors and missed updates

Chapter 3

Problem Statement, Objective & Scope

Problem Statement:

To design and build a centralized, secure and easy-to-use Campus Resource & Event Portal that supports smooth coordination of events, resource bookings and related communication by using modern web technologies like React, FastAPI and a relational database.

Description:

At present, the handling of campus events and resources is spread over several tools such as notice boards, email lists, messaging groups and separate online forms. Because these systems are isolated, organizers may unknowingly schedule overlapping events, duplicate the same data and struggle to keep everyone informed of changes. Participants may also receive incomplete or delayed information.

The proposed system replaces this scattered approach with one integrated portal. It connects event creation, approval, scheduling, resource allocation and student registration into a single workflow. All related data is stored in a common database, which makes records more consistent and easier to manage over time.

Objectives:

- To implement a role-based Campus Resource & Event Portal that can be used by admins, faculty coordinators and students.
- To simplify event creation, resource booking and notice publishing through guided online forms and automated checks.
- To protect data and enforce integrity by using a structured database and proper access control in the backend.
- To offer clear dashboards and views that improve user experience and make important information easy to find.
- To reduce manual work and scheduling issues by validating time slots and resource availability before confirming bookings.

Scope:

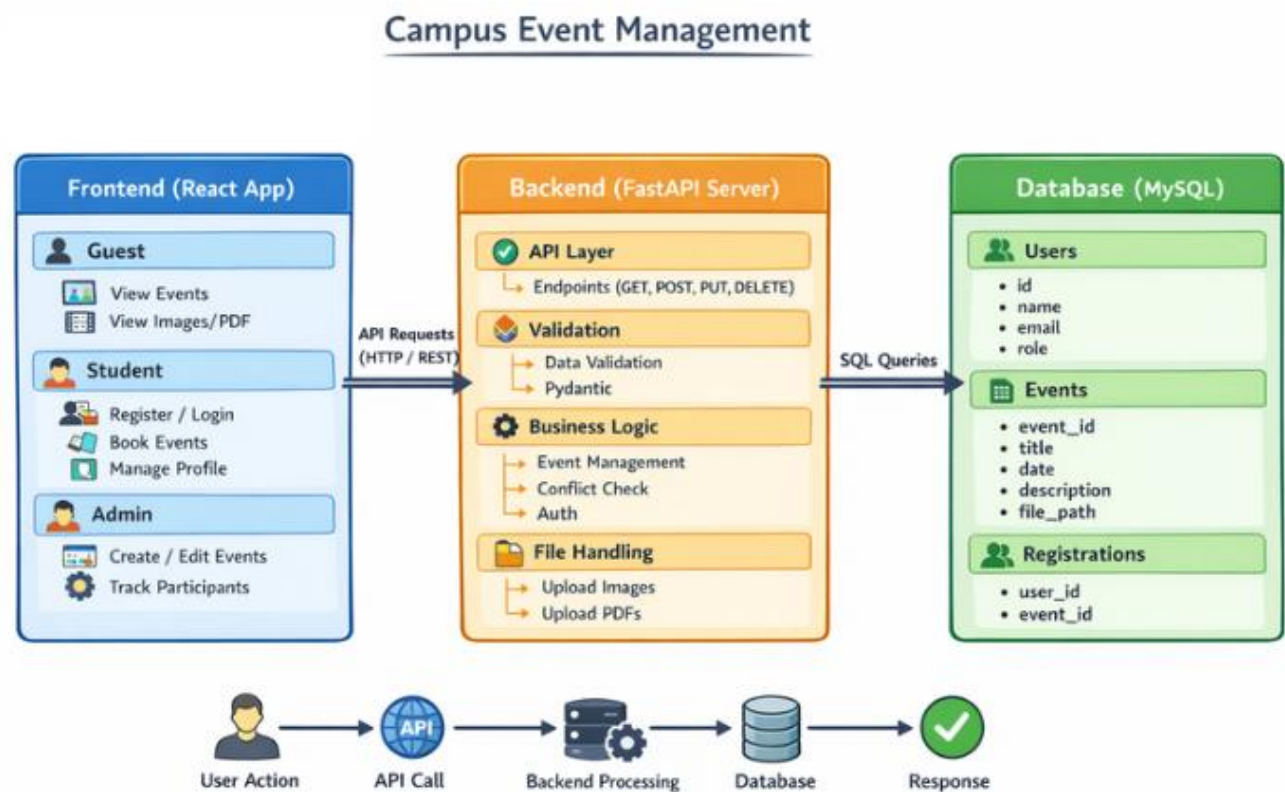
- The system focuses on academic and co-curricular events that take place within the campus.
- It supports main user roles such as Admin and Student, each with defined permissions and responsibilities.
- Admin-side features include creating, approving, editing and monitoring events and related announcements.
- Student-side features include browsing and searching events, viewing event details and registering for activities.
- A relational database is used to maintain structured records for users, events, resources, registrations and bookings.
- Basic checks help avoid double-booking of rooms or overlapping events for the same time and resource.
- Secure login and authorization logic control access to critical data and operations.
- The portal serves as a central hub for campus activity information and can be extended with new features in later phases.

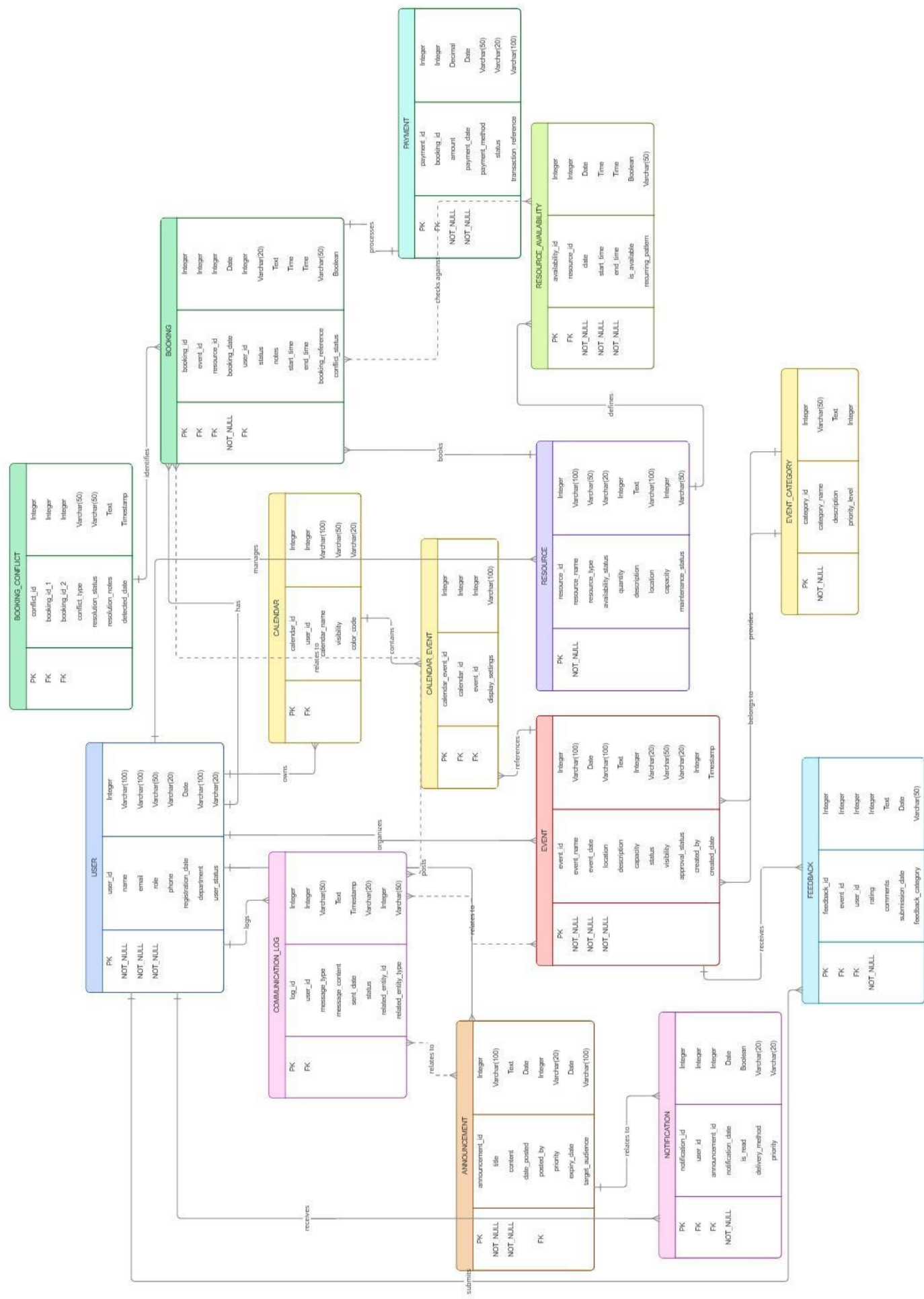
Chapter 4

Proposed System Architecture

Architecture diagram, ER diagram compulsory with explanation (each diagram on new page)

- Analysis/Framework/ Algorithm
- Design details
- Methodology (your approach to solve the problem) Proposed System





1. Core Entity: User

The USER table is central to the system.

Key attributes:

- user_id (PK)
- name, email, role, phone
- registration_date, department, status

What users do:

- Create and manage calendars
- Book resources
- Post announcements
- Send communications
- Receive notifications and give feedback

2. Calendar & Events System

CALENDAR

- Each user owns one or more calendars
- Stores visibility and display preferences

EVENT

Represents actual events.

Attributes:

- event_name, date, location

- capacity, status, visibility
- created_by (user)

CALENDAR_EVENT (junction table)

- Links events to calendars (many-to-many)
- Allows an event to appear in multiple calendars

EVENT_CATEGORY

- Categorizes events (e.g., meeting, workshop)
- Each event belongs to a category

3. Resource Management

RESOURCE

Represents assets like rooms, equipment, etc.

Attributes:

- resource_name, type
- location, capacity
- availability_status, maintenance_status

RESOURCE_AVAILABILITY

- Defines when a resource is available
- Includes date, start/end time, recurring patterns

4. Booking System

BOOKING

Central entity for reserving resources.

Attributes:

- booking_id, event_id, resource_id, user_id
- booking_date, start_time, end_time
- status, notes, reference
- conflict_status (boolean)

BOOKING_CONFLICT

- Detects and stores conflicts between bookings
- Links two conflicting bookings
- Tracks resolution status and notes

PAYMENT

- Associated with bookings
- Stores amount, payment method, status, transaction reference

5. Communication & Notifications

COMMUNICATION_LOG

Tracks messages sent by users.

Attributes:

- message_type, content
- sent_date, status
- related_entity_id + type (polymorphic link to events, bookings, etc.)

ANNOUNCEMENT

- Official messages posted by users
- Includes title, content, target audience, expiry

NOTIFICATION

- Sent to users (often triggered by announcements)
- Tracks delivery method, priority, read status

6. Feedback System

FEEDBACK

- Users provide feedback on events

Attributes:

- rating, comments
- submission_date, category

Key Relationships (Simplified)

User Relationships

- USER → CALENDAR (owns)
- USER → BOOKING (creates)
- USER → ANNOUNCEMENT (posts)
- USER → COMMUNICATION_LOG (logs messages)
- USER → FEEDBACK (gives feedback)

Event Relationships

- EVENT ↔ CALENDAR (via CALENDAR_EVENT)
- EVENT → EVENT_CATEGORY
- EVENT → FEEDBACK
- EVENT → BOOKING

Resource & Booking Relationships

- RESOURCE → BOOKING
- RESOURCE → RESOURCE_AVAILABILITY
- BOOKING → PAYMENT
- BOOKING ↔ BOOKING_CONFLICT

Relational Schema:

```
class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(..., min_length=6)
    full_name: str = Field(..., max_length=255)
    moodle_id: Optional[str] = Field(None, max_length=50)
    department: Optional[str] = Field(None, max_length=100)
    user_type: str = Field(..., pattern="^(student|admin)$") # 'student' or 'admin'

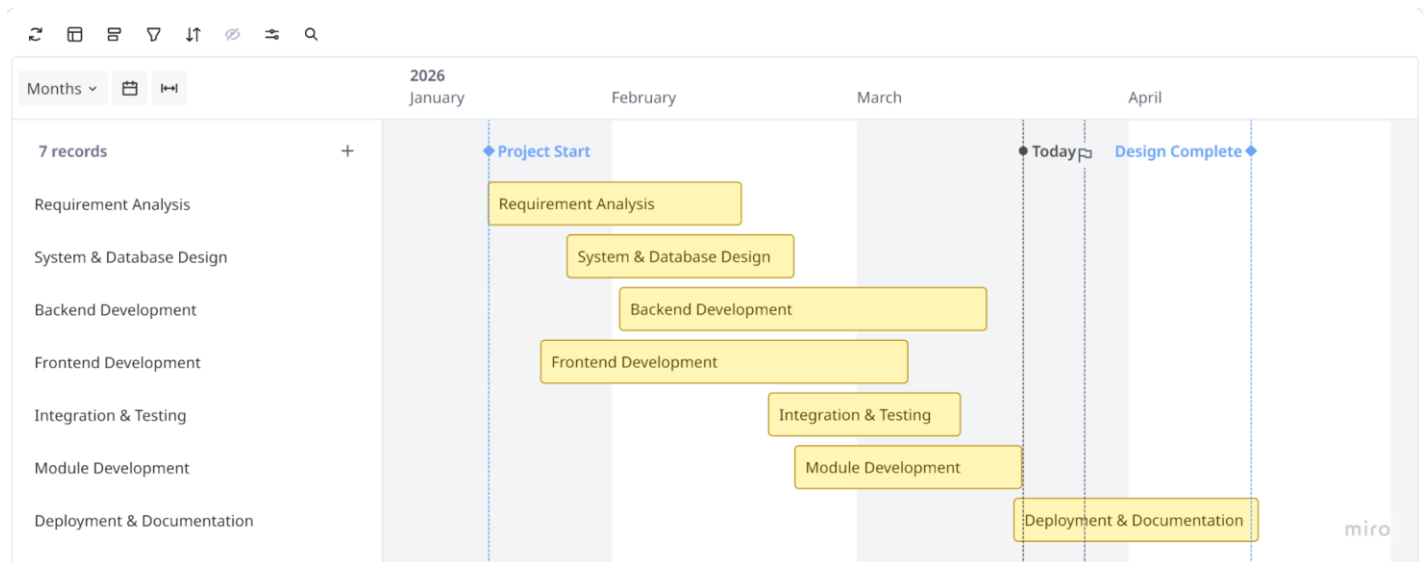
class UserLogin(BaseModel):
    email: EmailStr
    password: str
    # Explicit role requested at login; must match stored user_type
    user_type: str = Field(..., pattern="^(student|admin)$")

class UserUpdate(BaseModel):
    email: Optional[EmailStr] = None
    password: Optional[str] = Field(None, min_length=6)
    full_name: Optional[str] = Field(None, max_length=255)
    moodle_id: Optional[str] = Field(None, max_length=50)
    department: Optional[str] = Field(None, max_length=100)
    current_password: str # Required for verification
```

Chapter 5

Project Planning

Gantt chart



1. Requirement Analysis (Week 1–2)

Understand system needs: users, events, bookings, resources, and communication features.

2. System & Database Design (Week 3–4)

Create ER diagram (like yours), define schema, relationships, and system architecture.

3. Backend Development (Week 5–10)

Develop APIs and logic for:

- Users
- Events & calendars
- Resources
- Bookings & conflicts
- Payments

4. Frontend Development (Week 6–11)

Build UI for dashboards, booking forms, calendars, notifications, etc.

5. Module Development (Week 6–12)

Parallel development of key modules:

- User management
- Event handling
- Resource tracking
- Booking system
- Notifications & feedback

6. Integration & Testing (Week 11–14)

Combine frontend + backend, test APIs, fix bugs, validate workflows.

7. Deployment & Documentation (Week 15–16)

Deploy system and prepare user/admin documentation.

Chapter 6

Experimental Setup

6. Technology Stack Used

The Campus Resource & Event Portal is developed with a full-stack approach to ensure responsiveness, reliability and maintainability.

Frontend: The user interface is built using React.js, which enables creation of a dynamic and responsive layout. Its component-based structure allows the team to define reusable elements for headers, cards, forms and tables. React's state management helps in updating lists of events, registrations and notifications without reloading the page.

Backend: The backend layer is implemented in FastAPI, a Python framework well suited for building REST APIs. It is responsible for handling incoming requests, applying business rules, validating data and interacting with the database. FastAPI's design supports fast response times and automatic documentation of endpoints.

Database: Data is stored in MySQL, a relational database that organizes information into related tables. It holds user records, event details, resource information, booking entries and feedback data. Primary keys, foreign keys and constraints are used to maintain integrity and avoid inconsistent records.

Tools & Technologies:

- SQLAlchemy – used as the ORM layer for mapping Python objects to database tables and generating SQL queries.
- Postman – used during development to test API endpoints and debug request–response flows.
- Git & GitHub – used for version control and collaborative code management.
- Visual Studio Code – used as the main IDE for writing and organizing frontend and backend code.

2. System Environment

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: Minimum 4GB (8GB recommended)
- Storage: At least 500MB free space

Software Requirements:

- Operating System: Windows/Linux/macOS
- Web Browser: Google Chrome or Microsoft Edge
- Code Editor: Visual Studio Code
- Backend Server: Localhost (FastAPI server)

3. Database Design

The database is designed using a **relational model** to maintain structured and consistent data.

Key Tables:

- **User Table** – Stores user details such as ID, name, email, and role
- **Event Table** – Contains event information like title, date, venue, and category
- **Category Table** – Classifies different types of events
- **Participant Table** – Stores event registration details
- **Resource Table** – Maintains information about available resources
- **Booking Table** – Handles resource booking details

ER Diagram (Explanation):

- A **User** can create multiple **Events**
- An **Event** belongs to a **Category**
- A **User** can register for multiple **Events**
- A **Booking** connects **User**, **Event**, and **Resource**

Normalization:

The database follows **Third Normal Form (3NF)**:

- Eliminates redundant data
- Ensures all attributes depend on primary keys
- Improves data consistency and reduces anomalies

4. Module Description

The system is divided into different modules for better functionality and management.

1. User Management Module

- Handles user registration and login
- Provides role-based access (Admin, Faculty, Student)
- Ensures secure authentication

2. Event Management Module

- Allows creation, updating, and deletion of events
- Displays event details to users
- Categorizes events

3. Resource Management Module

- Manages campus resources such as rooms and equipment
- Tracks availability and usage

4. Booking Module

- Allows users to book resources
- Prevents double booking using validation
- Maintains booking records

5. Notification Module

- Provides updates about events and bookings
- Displays announcements

6. Feedback Module

- Collects feedback from users
- Stores ratings and suggestions

5. Working Procedure

1. User logs into the system
2. Admin or faculty creates an event
3. System checks resource availability
4. Users view and register for events
5. Booking is validated and stored in the database
6. Notifications are sent to users
7. Feedback is collected after event completion

6. Testing

The system is tested using a combination of manual and tool-assisted techniques to ensure correctness, reliability, and usability,

- **Unit Testing:**

Individual functions and modules (such as user authentication, event creation, and booking logic) are tested in isolation to verify their correctness,

- **Integration Testing:**

Ensures that different modules, as well as frontend and backend components, work together correctly and exchange data as expected.

- **API Testing:**

Backend APIs are tested using Postman to check request/response formats, status codes, validation rules, and error handling

User/Functional Testing:

End-to-end workflows are executed from a user perspective to validate usability and functionality,

Sample Test Cases Include:

- User login and registration with valid and invalid data.
- Event creation, updating, and display on the portal
- Resource booking with validation of required fields,
- Conflict detection when two bookings overlap for the same resource.

7. Performance Considerations

The system is designed with performance and scalability in mind,

- Fast response times are achieved using Fast API's asynchronous request handling and optimized API endpoints.
- Efficient database queries and indexing strategies help reduce query execution time, especially for events and bookings.
- Database normalization reduces redundancy, which in turn improves consistency and storage efficiency:
- The overall architecture supports horizontal and vertical scaling to handle an increasing number of users, events, and concurrent requests as the system grows,

Chapter 7

Implementation Details

1. Frontend–Backend Integration

The project follows a full-stack architecture where the frontend and backend communicate through RESTful APIs. The frontend, developed using React and Vite, interacts with the backend by sending HTTP requests using the Fetch API. These requests are directed to FastAPI endpoints hosted at `http://localhost:8000/api/...`

User actions such as login, registration, event browsing, and participation trigger API calls from the frontend. The backend processes these requests, performs necessary operations, and returns responses in JSON format. The frontend then dynamically updates the user interface based on the received data, ensuring a seamless and real-time user experience.

2. Database Integration

The backend is integrated with a MySQL relational database using SQLAlchemy ORM. The database stores structured data related to users, events, categories, and participants.

- User Table: Stores user credentials and role information
- Event Table: Stores event details such as title, date, venue, and category
- Category Table: Manages classification of events
- Participant Table: Maintains records of user registrations for events

SQLAlchemy manages the connection, query execution, and data mapping between Python objects and database tables. This ensures efficient data handling, consistency, and scalability.

3. SQL Queries & Implementation

The system performs various database operations using SQL queries generated through SQLAlchemy ORM. These include:

- Insertion Queries:
Used for adding new users, events, and participant registrations
(*e.g., registering a new user or creating an event*)

- Selection Queries:
Retrieve data such as event lists, user details, and participant records
(e.g., *fetching all events or events by category*)
- Update Queries:
Modify existing records like updating user profiles or event details
- Deletion Queries:
Remove events or user records when required
- Validation Queries:
Ensure no duplicate bookings or event conflicts by checking date and time conditions before insertion

4. Implementation Workflow

- The user interacts with the frontend (e.g., registers for an event).
- The frontend sends a request to the backend API.
- FastAPI processes the request and interacts with the database using SQLAlchemy.
- The database executes the query and returns the result.
- The backend sends a response to the frontend.
- The frontend updates the UI accordingly.

5. Key Features of Integration

- Real-time data fetching and updates
- Secure data handling through API-based communication
- Efficient database operations using ORM
- Prevention of event conflicts through validation logic
- Smooth user experience with dynamic UI updates

Chapter 8

Result

SDG 4 – Quality Education

The portal supports quality education by making information about seminars, workshops and academic events easier to access. Students can clearly see upcoming opportunities and register on time, which helps them build skills beyond regular classroom teaching.

SDG 9 – Industry, Innovation & Infrastructure

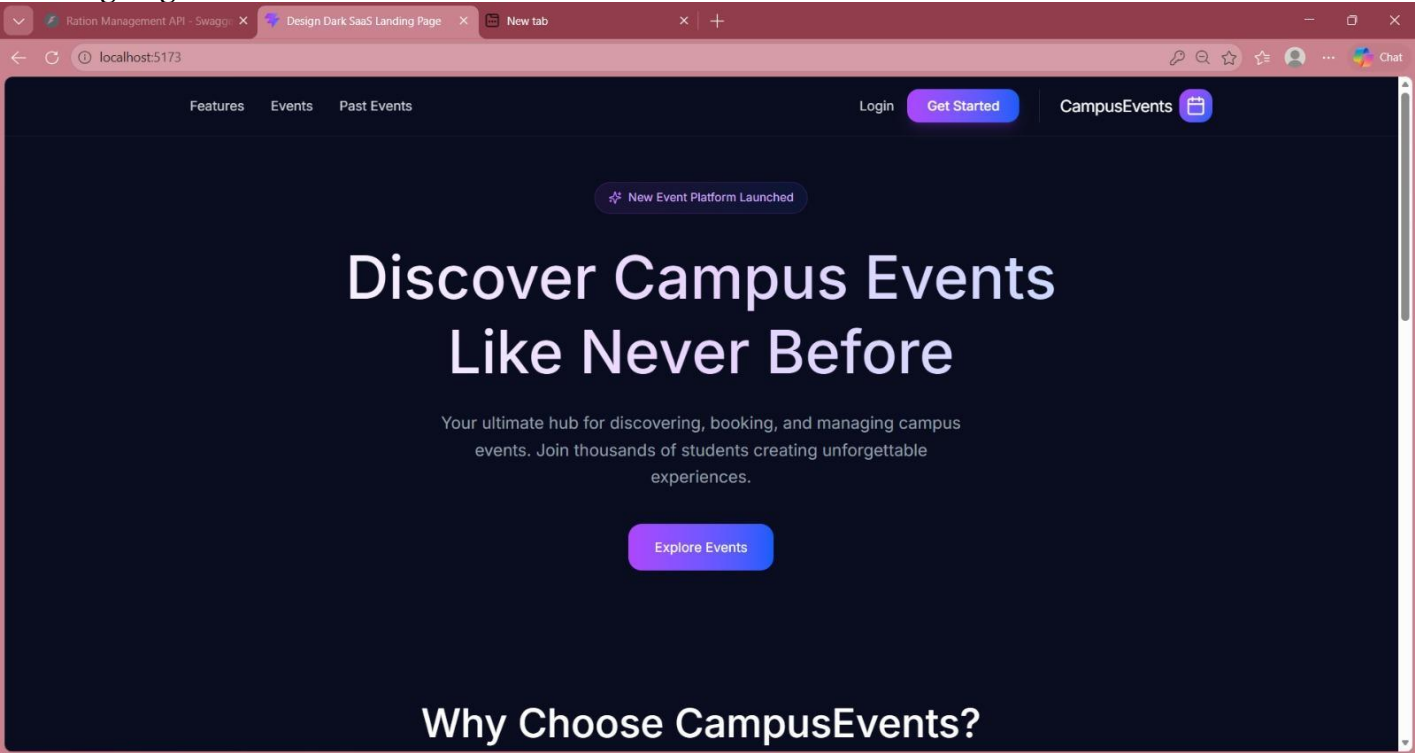
By using a web-based system that integrates modern tools and frameworks, the project upgrades the way the institute manages internal processes. It encourages a shift from manual, paper-based methods to an innovative, technology-driven infrastructure that is easier to scale.

SDG 16 – Peace, Justice & Strong Institutions

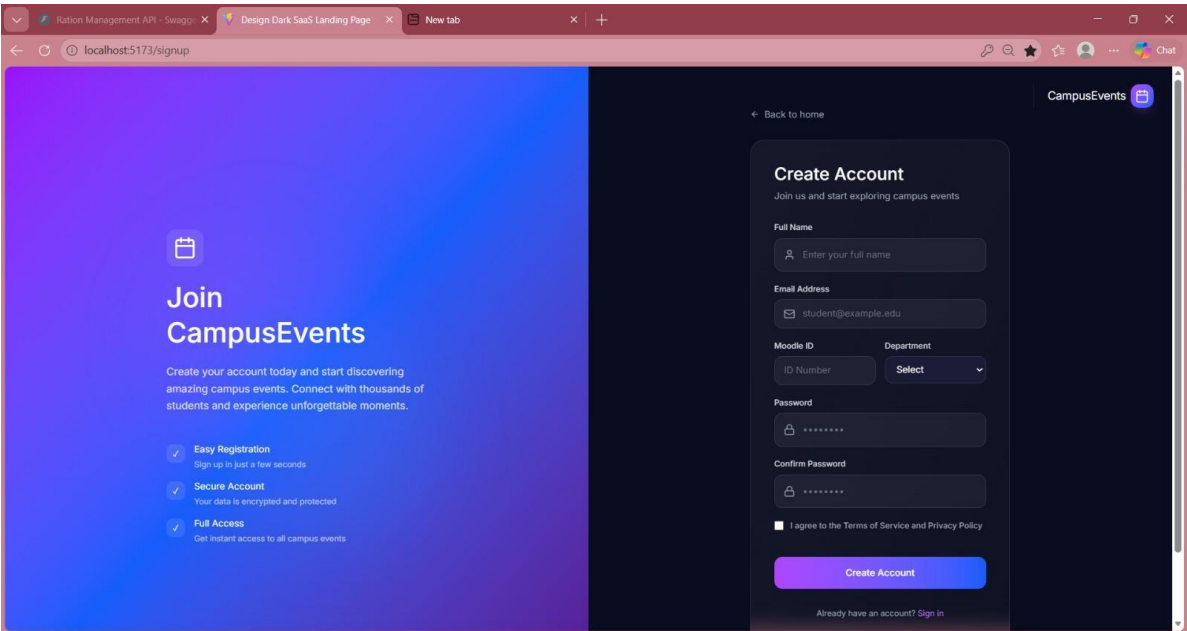
The portal improves transparency in how events and resources are managed because actions are recorded and follow defined rules. Role-based permissions and consistent workflows reduce confusion and help build trust among students, staff and administrators.

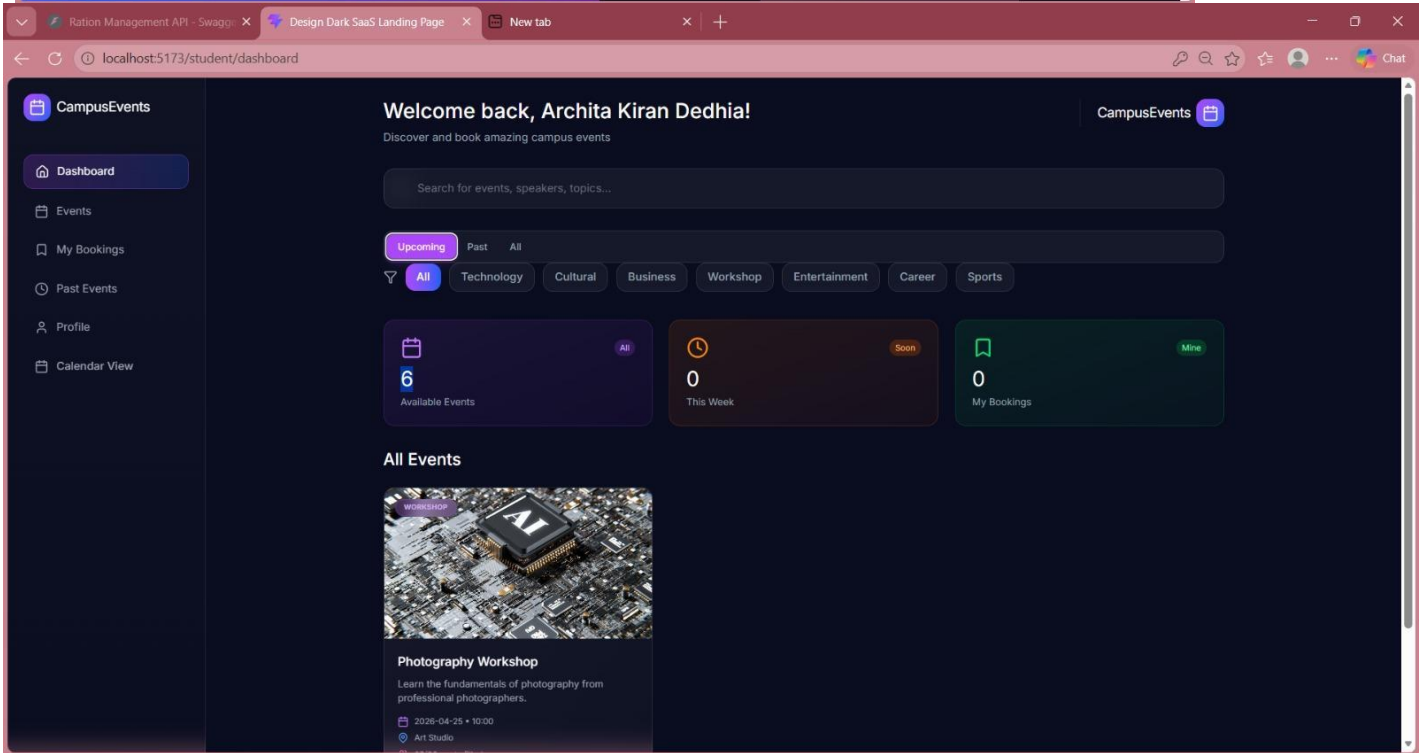
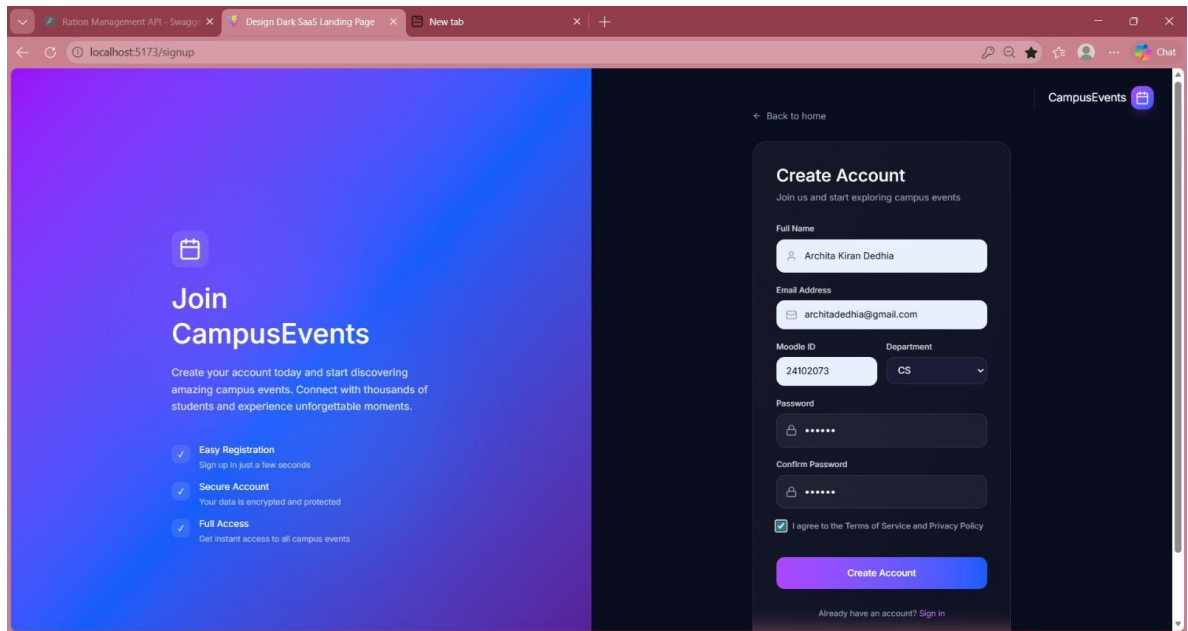
Frontend:

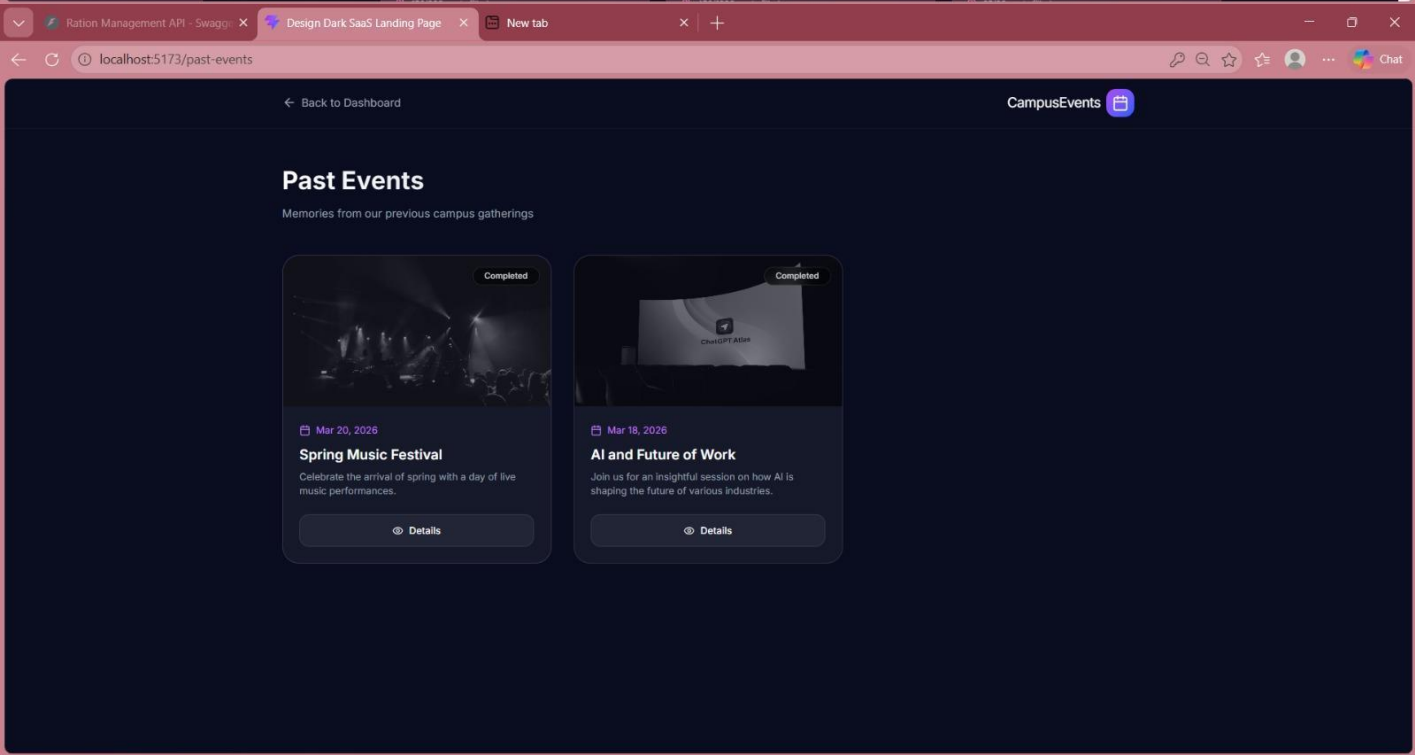
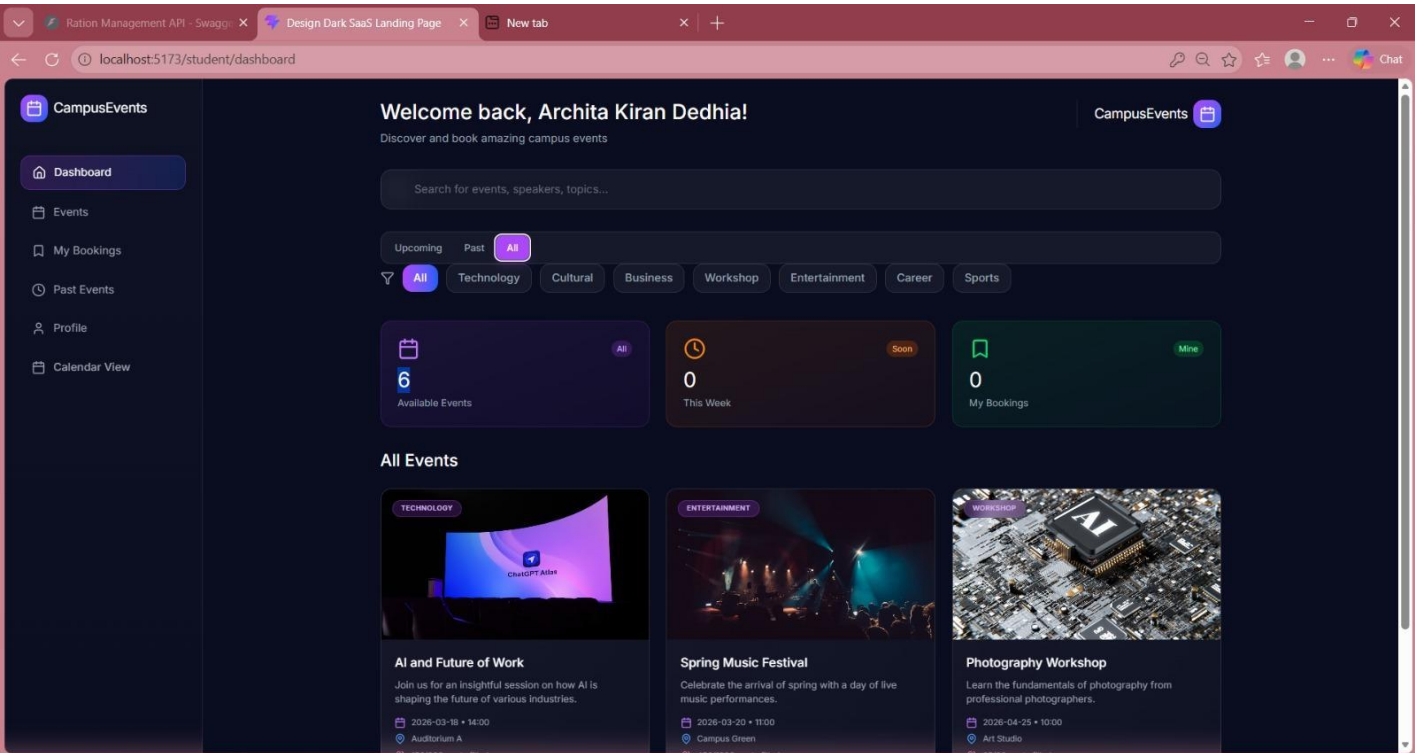
Landing Page:

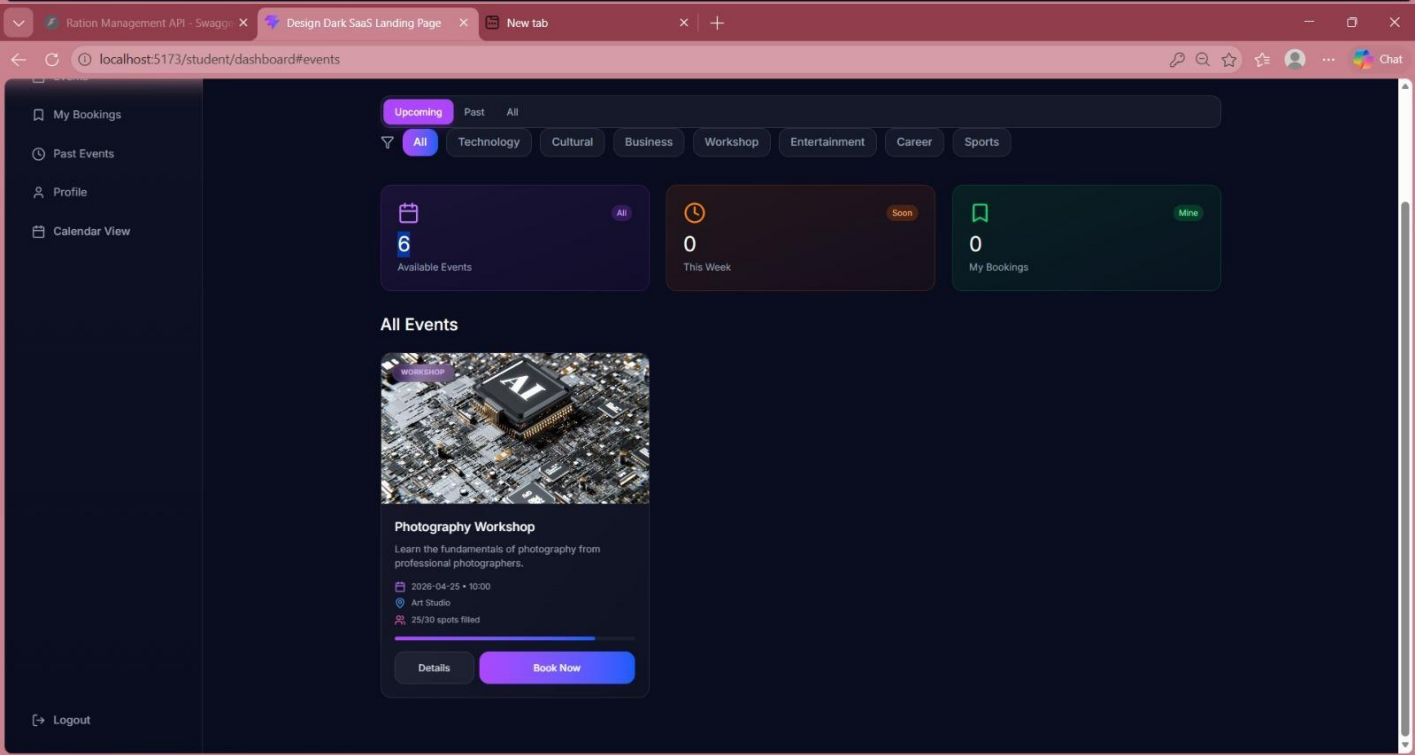
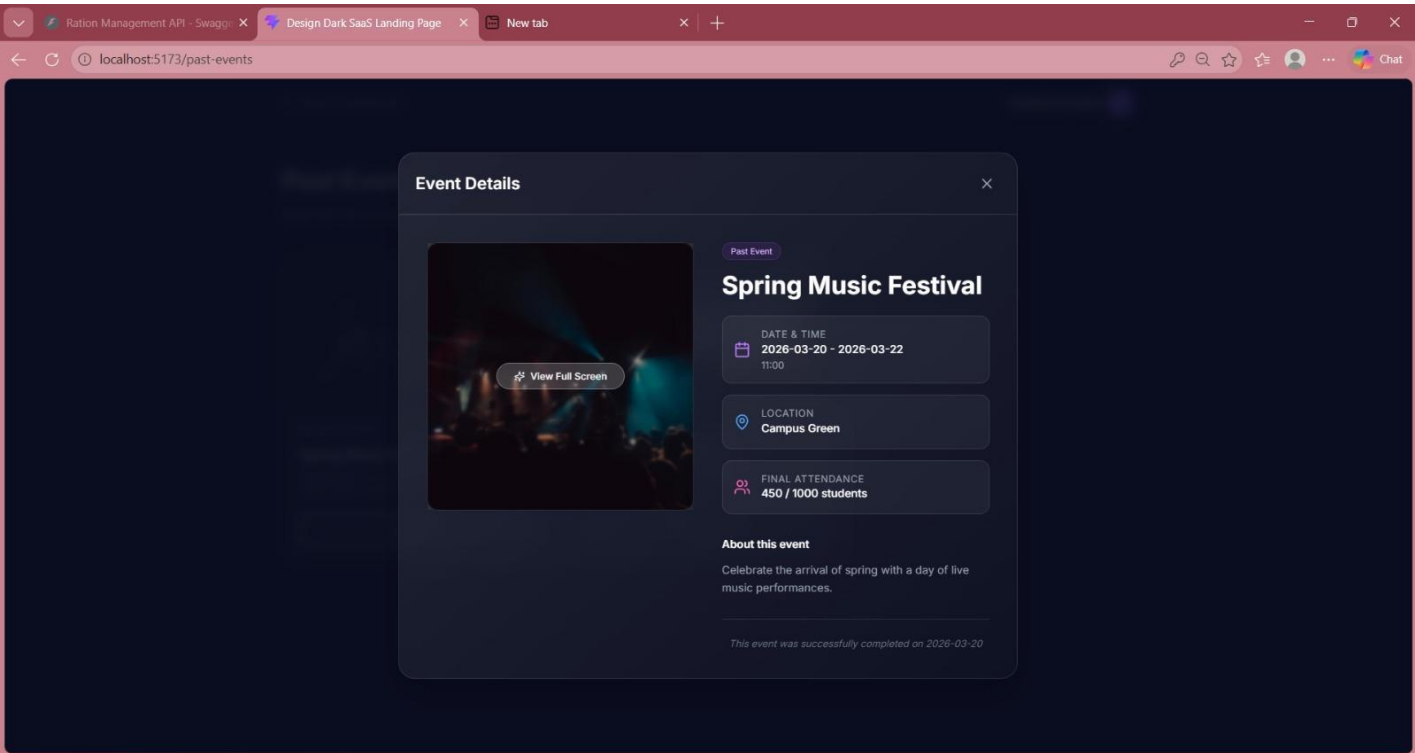


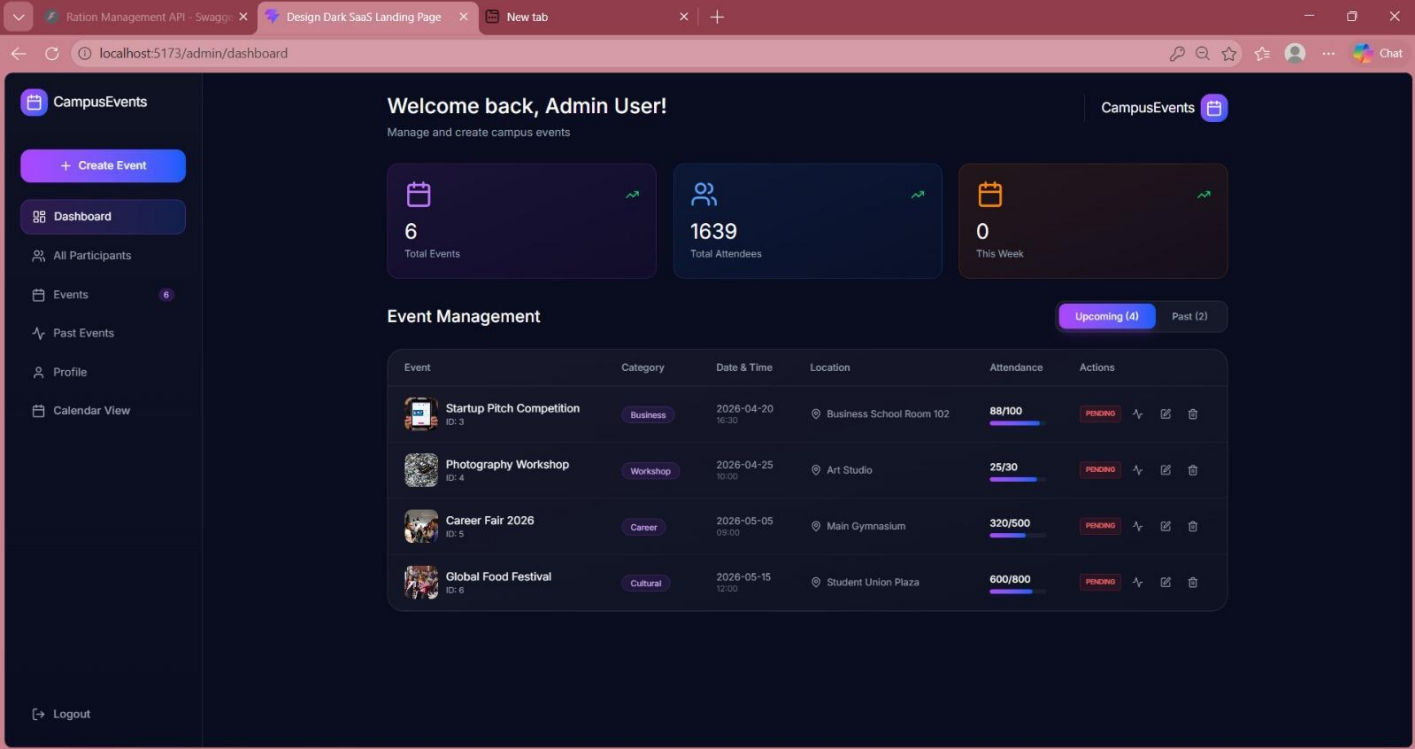
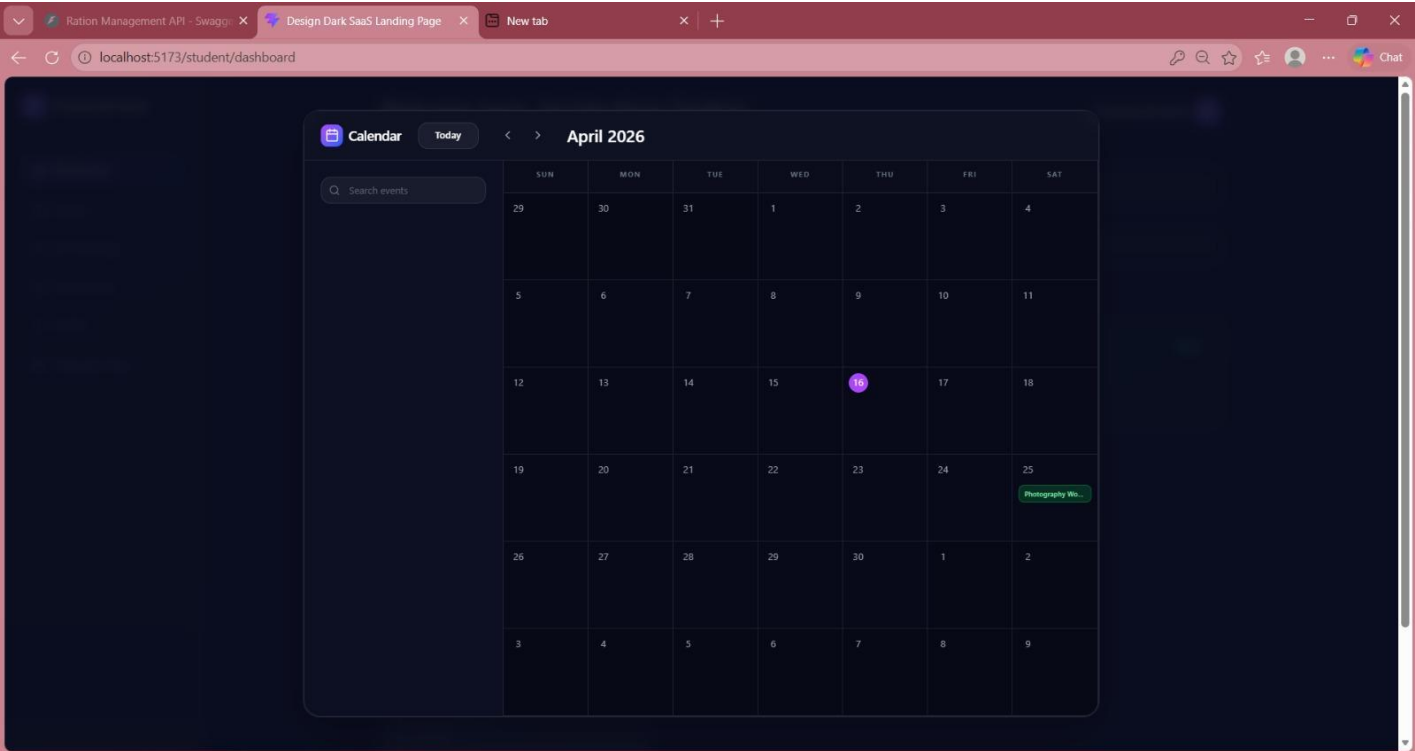
Login Page:

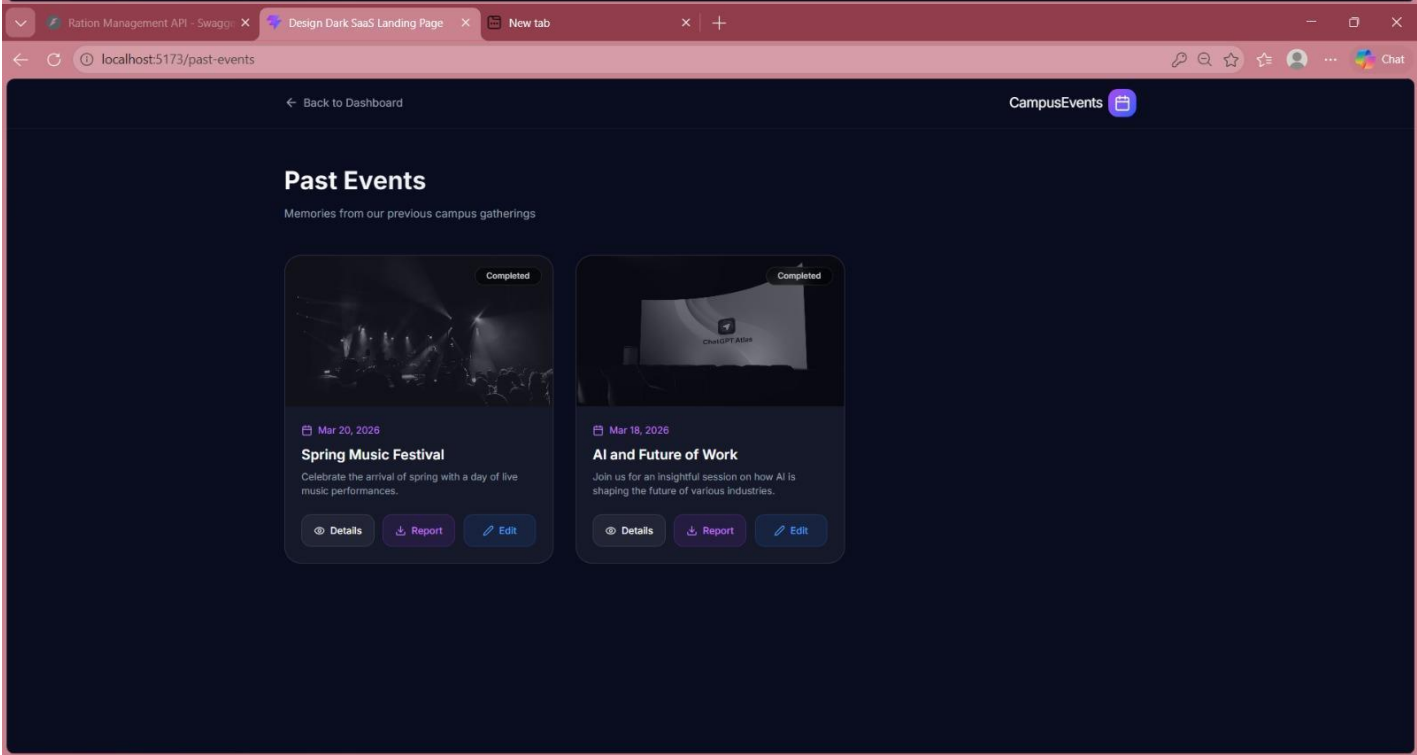
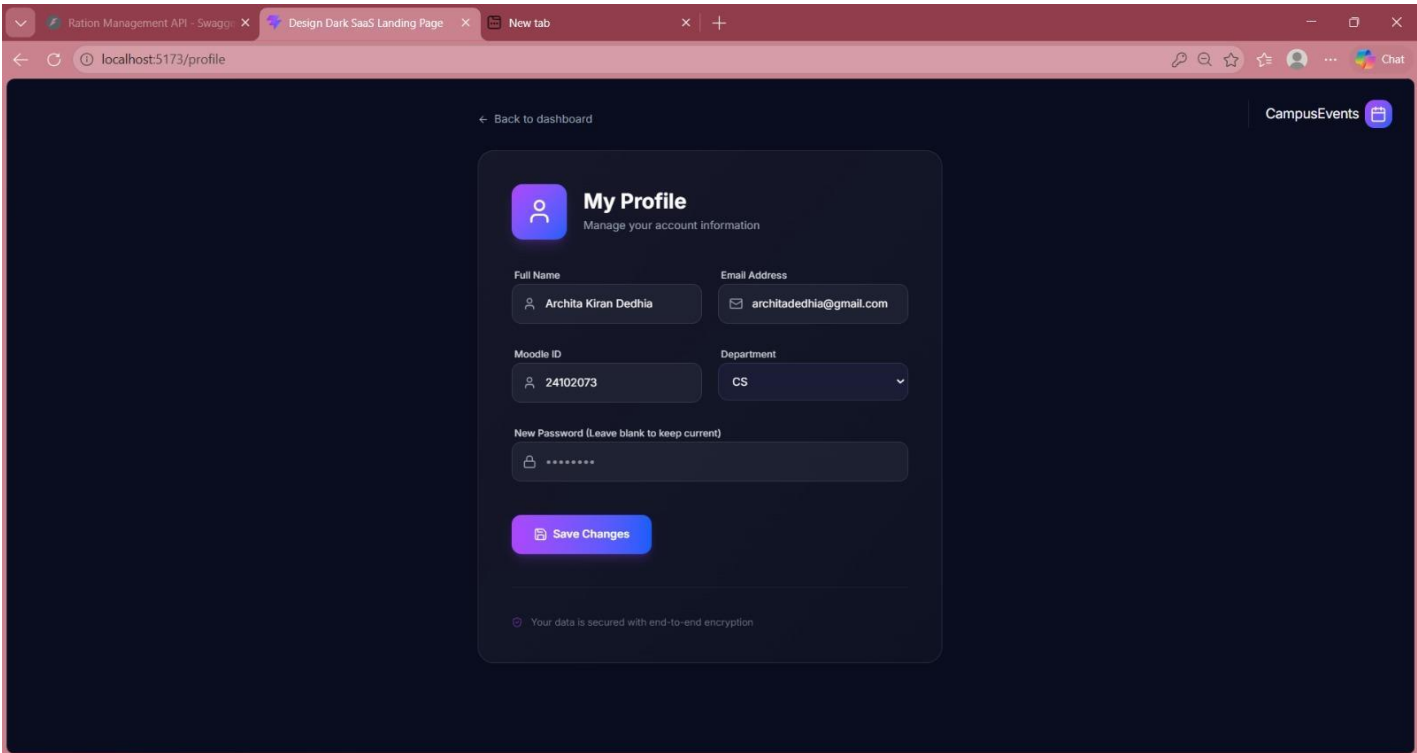




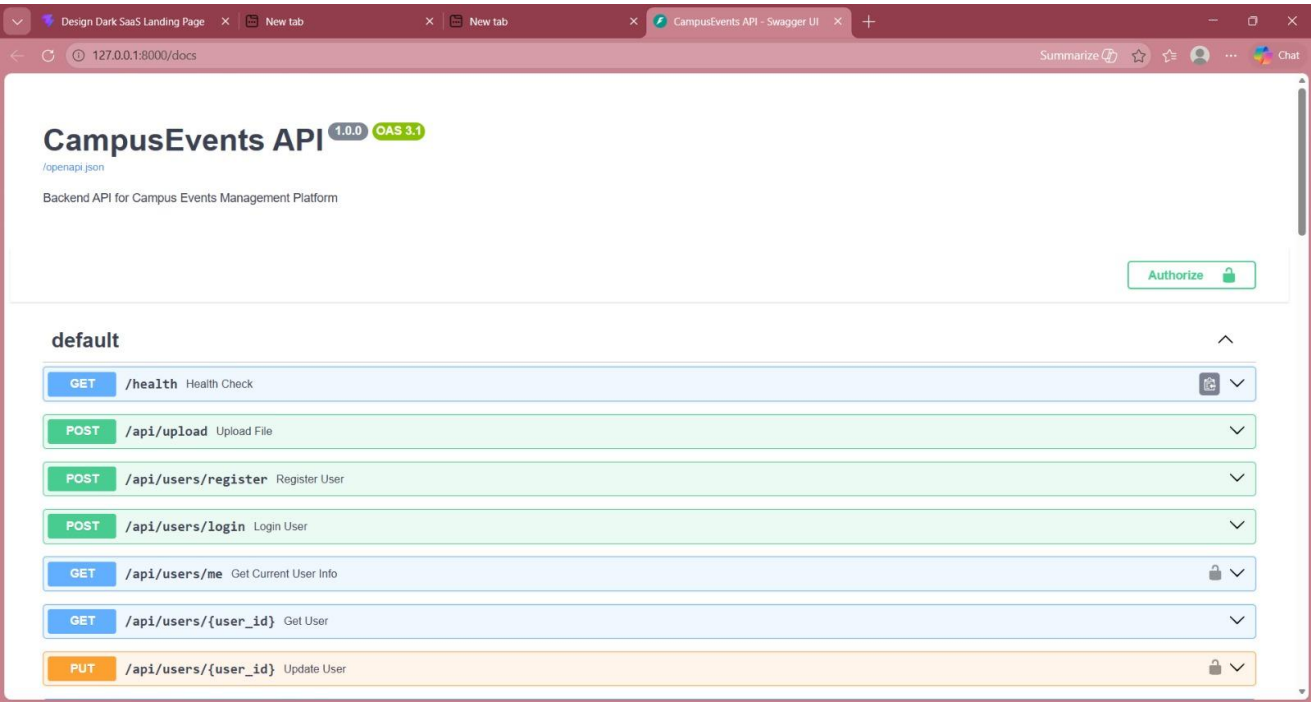
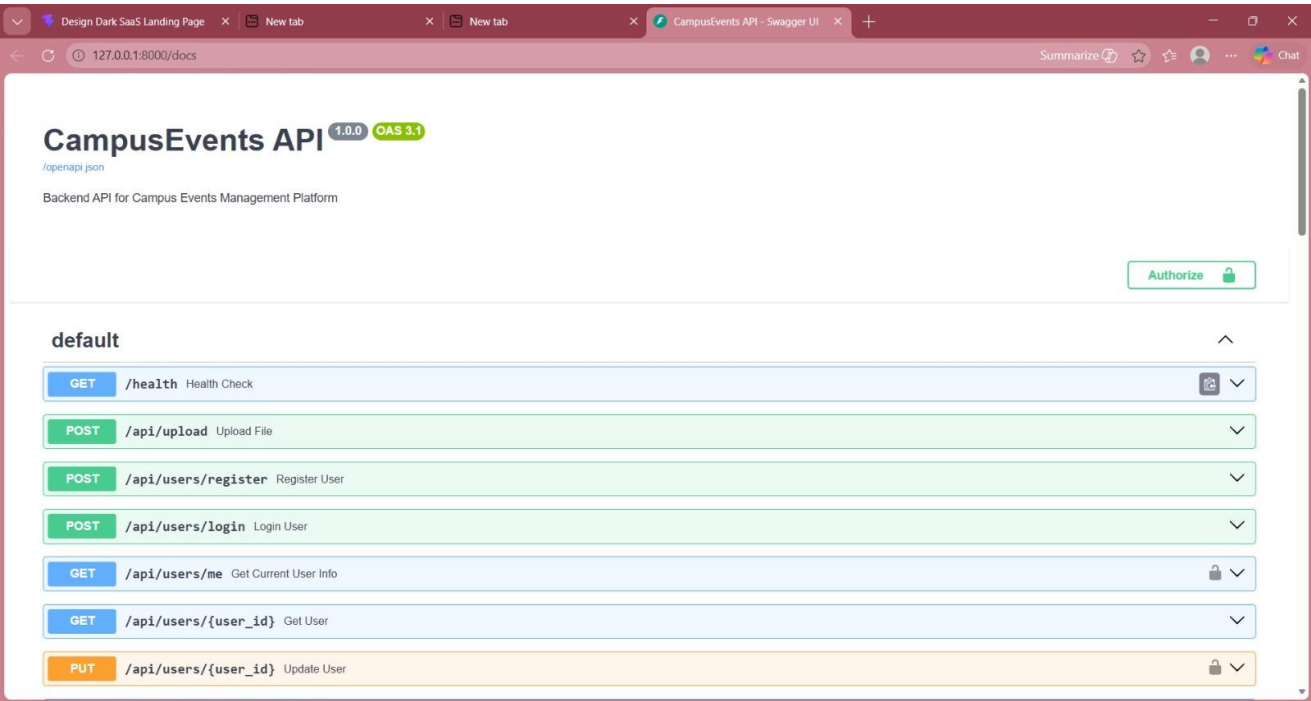








Backend:



Design Dark SaaS Landing PageNew tabNew tabCampusEvents API - Swagger UI

127.0.0.1:8000/docs

SummarizeStarFavoriteUserMoreChat

Authorize

GET/api/participants/user/{user_id}Get User Events

GET/api/participants/event/{event_id}Get Event Participants

DELETE/api/participants/{participant_id}Cancel Booking

GET/api/admin/participantsGet All Organizer Participants

GET/api/admin/analyticsGet Admin Analytics

Schemas

AdminAnalytics > Expand all object

Body_upload_file_api_upload_post > Expand all object

CategoryCreate > Expand all object

CategoryOut > Expand all object

EventCreate > Expand all object

Design Dark SaaS Landing PageNew tabNew tabCampusEvents API - Swagger UI

127.0.0.1:8000/docs

SummarizeStarFavoriteUserMoreChat

Authorize

EventCreate > Expand all object

EventDetail > Expand all object

EventImageOut > Expand all object

EventOut > Expand all object

EventUpdate > Expand all object

HTTPValidationError > Expand all object

LoginResponse > Expand all object

MessageResponse > Expand all object

ParticipantCreate > Expand all object

ParticipantDetail > Expand all object

ParticipantOut > Expand all object

Design Dark SaaS Landing PageNew tabNew tabCampusEvents API - Swagger UI

127.0.0.1:8000/docsSummarizeStarBookmarkUserAvatarChat

Authorize

MessageResponse > Expand all object

ParticipantCreate > Expand all object

ParticipantDetail > Expand all object

ParticipantOut > Expand all object

UserCreate > Expand all object

UserDetail > Expand all object

UserLogin > Expand all object

UserOut > Expand all object

UserUpdate > Expand all object

ValidationError > Expand all object

Chapter 9

Conclusion

The Campus Resource & Event Portal tackles key problems seen in traditional campus event management, where information and bookings are spread across many unconnected tools. By offering a single, web-based platform, it brings together event details, registrations and basic resource handling, making everyday coordination more straightforward.

Through role-based access, the portal assigns clear responsibilities to administrators, faculty and students. Admins can create and supervise events, staff can coordinate specific activities and students can quickly find and join events that interest them. This clarity leads to smoother communication and better use of campus facilities.

The combination of React.js, FastAPI and MySQL provides a solid technical base for the system. Features like real-time interface updates, structured data storage and validation checks for bookings contribute to a more reliable user experience. Conflict-prevention logic further reduces errors in scheduling.

By automating repetitive tasks and centralizing data, the portal reduces manual workload for organizers and minimizes the chance of missed announcements. It improves overall coordination and supports a more organized campus environment. As a result, the project offers a practical solution that can grow with future requirements and enhancements.

Chapter 10

References

- Chaturvedi, A., Sharma, K., Dua, A., & Gupta, A. (2024). CU-Events: A Comprehensive Event Management System for University. International Journal for Research in Applied Science & Engineering Technology (IJRASET).
- Rakesh, K., Sai Vishnu, K., & Nikhith Reddy, G. (2025). Unified Web Platform for Student-Campus Connectivity: A React.js-Based Approach. IJSDR (International Journal of Scientific Development and Research).
- Authors as per IJRPR publication. (2024). EduLearn: A React Based EdTech Web Application. International Journal for Research Publication & Review (IJRPR).
- Chaturvedi, A., Sharma, K., Dua, A., & Gupta, A. (2024). College Event Management System. International Journal of Engineering Science & Advanced Technology (IJESAT).